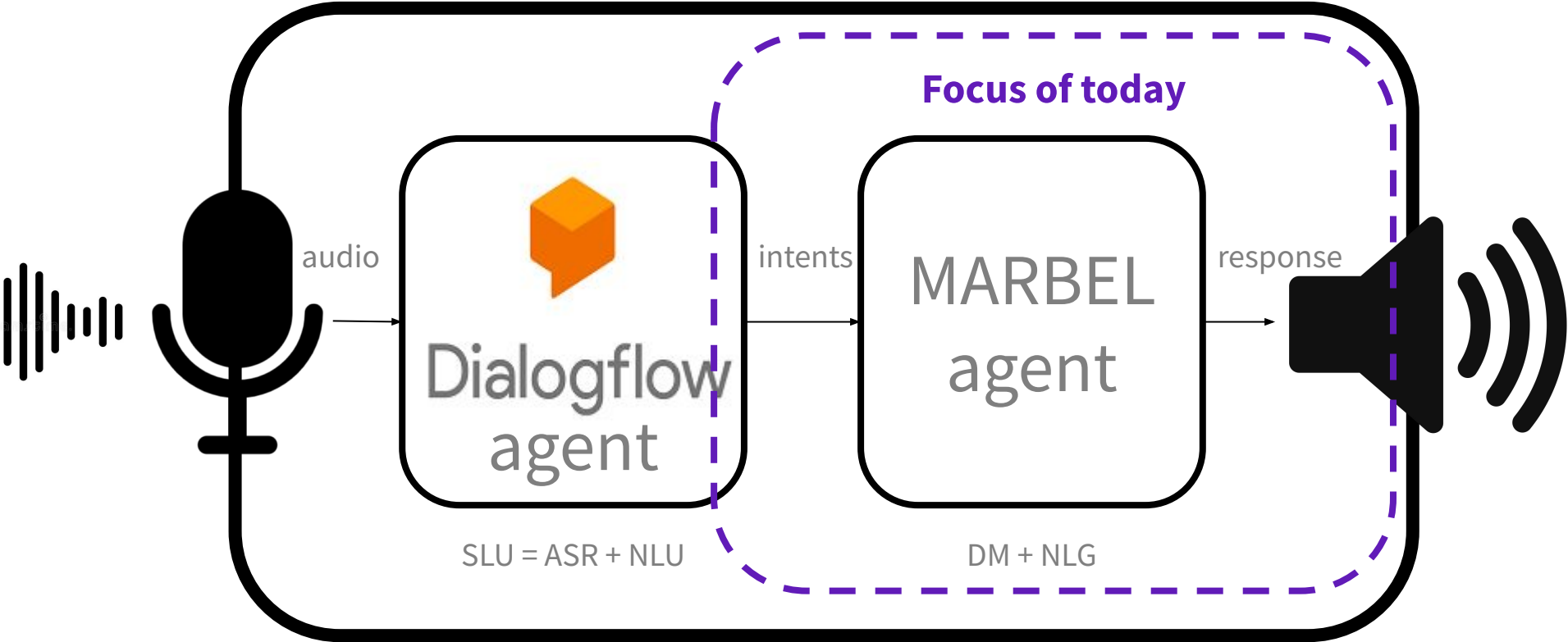


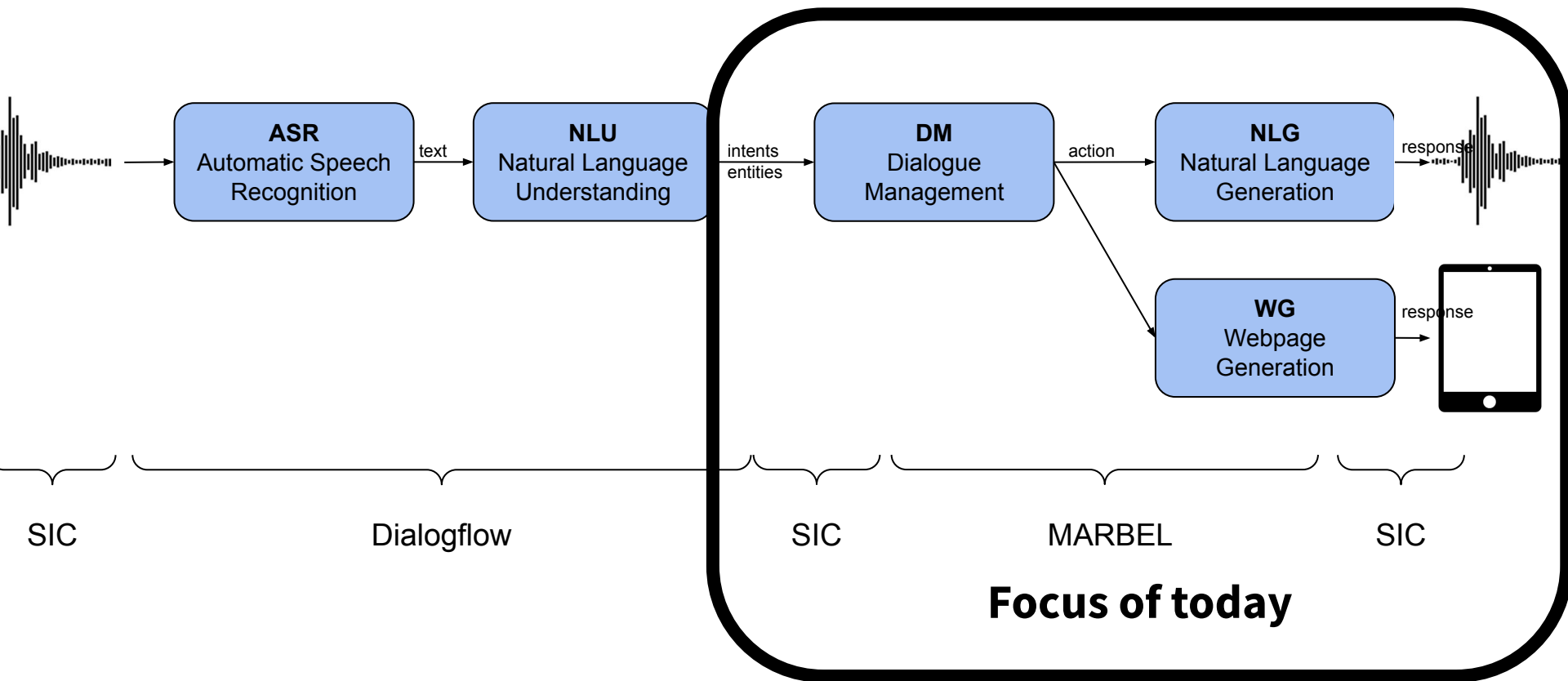
Conversational Patterns in Language Generation

Project Conversational Agents - Lecture 3

Our conversational agent is built out of other agents



Pipeline Architecture



Overview

- **Architecture**
 - Dialogflow-MARBEL Connection
 - Patterns
 - Repair
 - Responses
 - Visuals
 - Summary
-

The PCA project builds on various components

- **Redis**
- **Dialogflow** (connecting to a Dialogflow agent)
- **Webserver** (Flask)
- **EISComponent** (start framework)
- **MARBEL agent**

Why so many components?

Requirement 1: Speech based interaction (SLU, NLG)

Requirement 2: Only recipes from our database (company RQ)

Requirement 3: Implement business logic (customer preferences)

Requirement 4: Visuals (display relevant info to customer)

Why so many components?

Requirement 1: Speech based interaction (SLU, NLG)

How can we meet this requirement?

- ChatGPT would work (can do NLU, NLG, speech);
- **PCA26:**
 - Dialogflow / Whisper+NLU &
 - MARBEL agent + eSpeak / TTS Google

Why so many components?

Requirement 2: Only recipes from our database (company RQ)

How can we meet this requirement?

- Use a Retrieval-Augmented Generation (RAG) component
- **PCA26:**
 - MARBEL agent

Why so many components?

Requirement 3: Implement business logic (customer preferences)

How can we meet this requirement?

- Use a Memory Module (to keep track of selected criteria)
- **PCA26:**
 - MARBEL agent

Why so many components?

Requirement 4: Visuals (display relevant info to customer)

How can we meet this requirement?

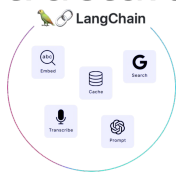
- Use a presentation layer
- **PCA26:**
 - A webserver (Flask/HTML)

Why so many components?

Possible alternative pipeline

- ChatGPT
- RAG component
- Memory module
- Presentation layer
- Infrastructural components,

e.g.

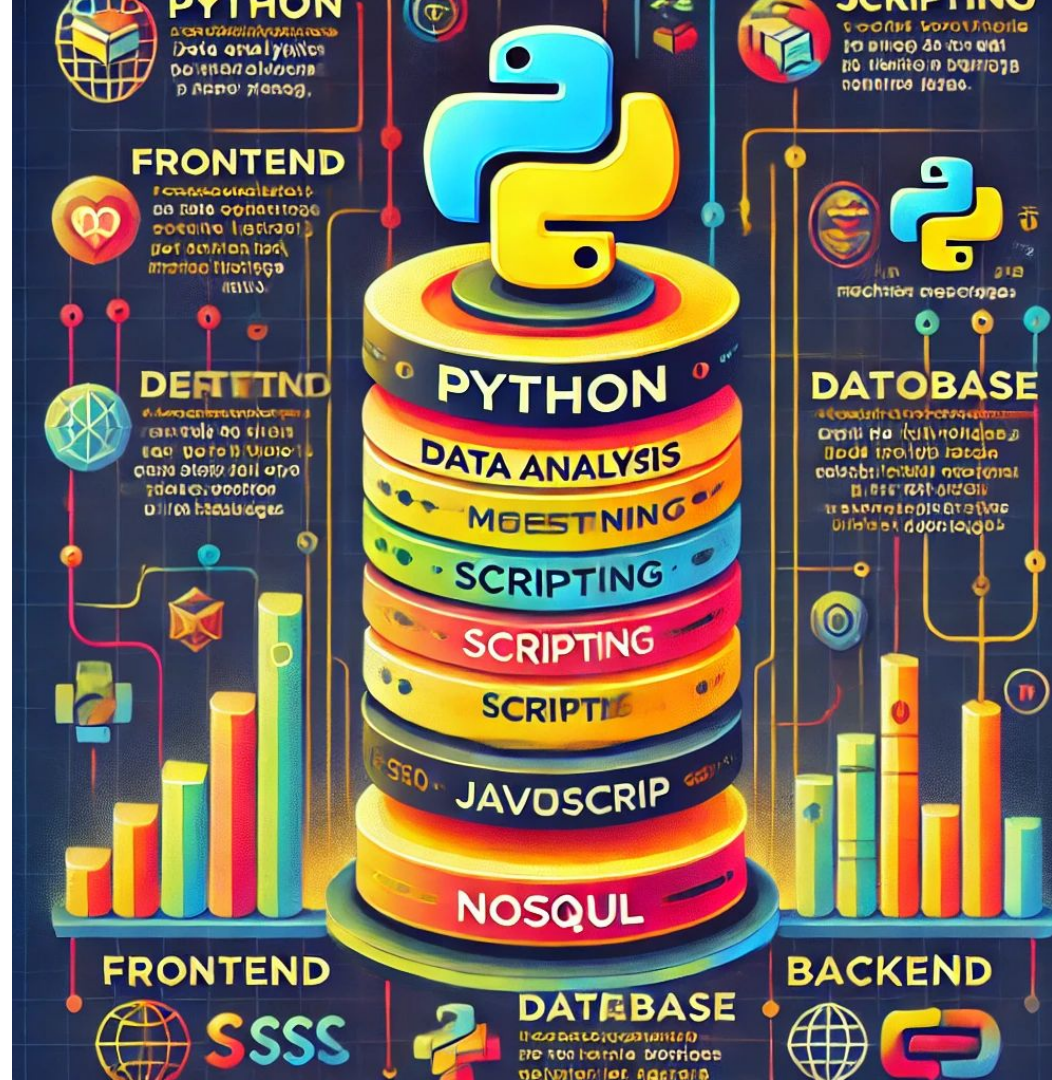


PCA26

- Dialogflow / Whisper+NLU
- eSpeak / TTS Google
- MARBEL agent
- Webserver (Flask+HTML)
- Infrastructure (SIC/Redis + EIS)

**Control &
Explainability**

Python vs Technology stack



PCA project: functional perspective on components

Functionalities we need:

- **Dialogflow:** SLU = ASR + NLU
- **Webserver** (Flask): Visualization
- **MARBEL agent:** Dialogue management + Database logic

Infrastructure: (glue between components)

- **Redis:** Pub/sub architecture for messaging between components
- **EISComponent:** Dedicated component for connecting MARBEL agent

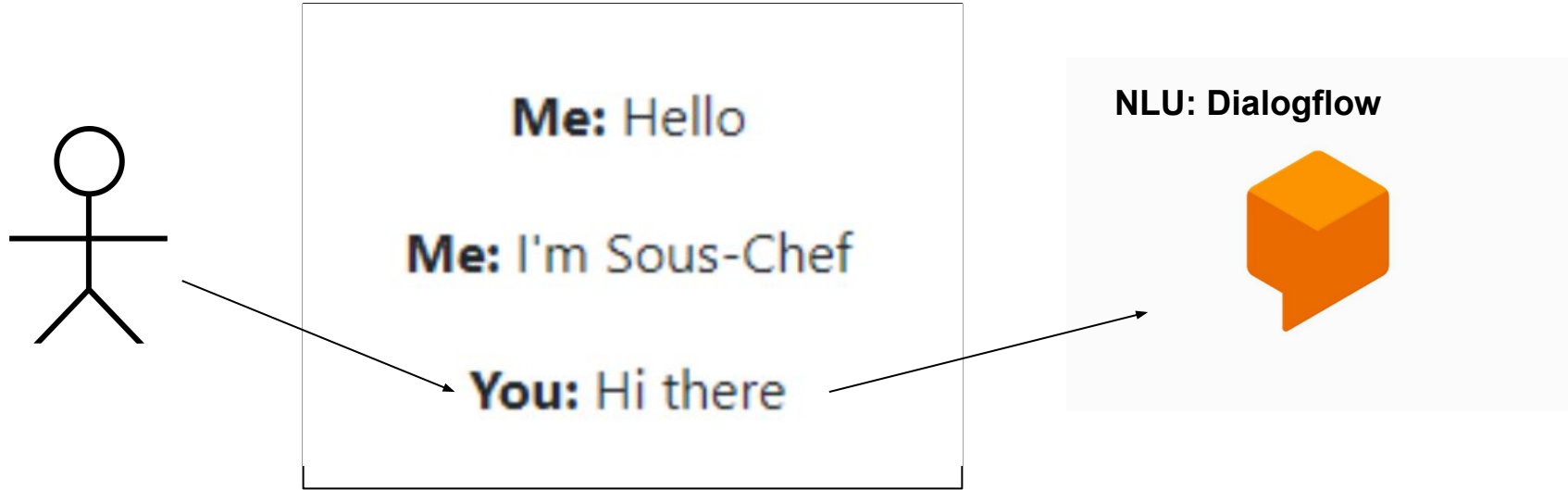
Overview

- Architecture
 - **Dialogflow-MARBEL Connection**
 - Patterns
 - Repair
 - Responses
 - Visuals
 - Summary
-

MARBEL agent: functional perspective

- Dialogue management (approach called SUPPLE):
 - Using conversational patterns;
 - includes interaction management;
- Natural Language Generation (NLG): scripted text;
- Recipe database
 - Retrieval
 - Filtering (of recipes)

SUPPLE Dialog Management - Flow



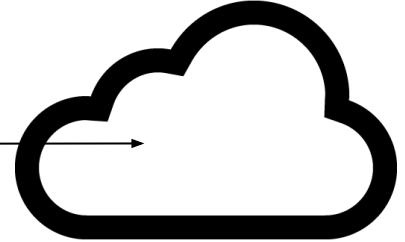
SUPPLE Dialog Management - Flow

NLU: Dialogflow

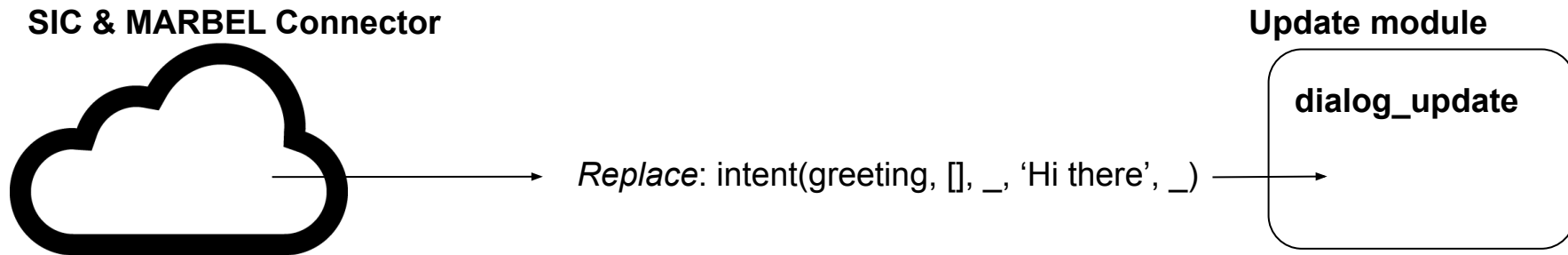


Intent: greeting (100%)
Entities: -

SIC & MARBEL Connector



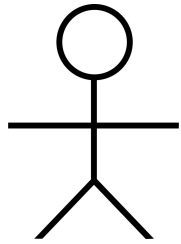
SUPPLE Dialog Management - Flow



Predicate: **intent(Intent, Parameters, Confidence, Transcription, Source)**

Agent also receives Dialogflow confidence score (1-100) and the source (e.g., 'speech', 'chat', 'touch').

SUPPLE Dialog Management - Flow



NLU: Dialogflow



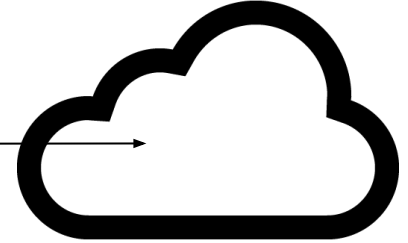
SUPPLE Dialog Management - Flow

NLU: Dialogflow



Intent: addFilter
Entities: ingredient = [pasta]

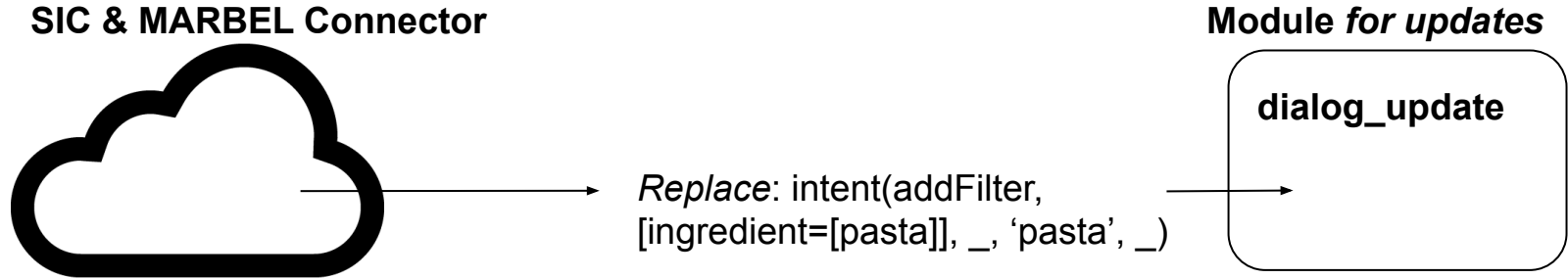
SIC & MARBEL Connector



Reference value of an entity in your Dialogflow agent

Parameter name in your Dialogflow agent

SUPPLE Dialog Management - Flow



Module *for updates*

`dialog_update.mod2g`

1. Session updating (see Patterns section)
2. Logging (files in MARBEL log folder)
3. Event processing: e.g., button handling

4. Event processing: Intents

5. Conversation management
 6. Agenda handling
-

What happens when an intent is received?

The dialog manager (using `dialog_update.mod2g` module):

1. The system will have stopped listening (when an intent is received or not); an event (`TextDone`) will be sent, which is handled by the module;
2. Checks for issues with intent:
 - a. No intent label? → add fallback intent 'input.unknown' as label to intent
 - b. Empty parameter lists? → replace intent with fall back intent (*assuming some parameter is required*)
3. Preprocesses parameters:
 - a. E.g., Dialogflow parameter `duration = [ "unit": "min", "amount": 45 ]` mapped to `duration = 45`.
4. Updates the agent's **conversational memory**.
5. Updates the agent's session (conversational history).

Conversational memory

Dialog manager keeps track of parameter-value pairs received from Dialogflow agent.

Example:

```
[ingredient=pasta, ingredient=garlic, button=start]
```

NB: parameter values are preprocessed! (see previous slide) before put into memory.

Memory Update Logic

Updated every time when a new parameter-value pair is received *with an intent*.

Uses logic defined in `dialog.pl`:

```
updateMemory(NewParams, OldMemory, NewMemory) :-  
    memory(OldMemory),  
    removeEmptyParams(NewParams, Params),  
    setof(Key, V^member(Key=V, Params), Keys),  
    remove(Keys, OldMemory, CleanedParams),  
    append(Params, CleanedParams, NewMemory).
```

“Overwrite” logic: old values replaced with new ones.

Memory: Update Logic (*Simplified Example!*)

```
update([ingr=garlic, utensil=''], [recipe=pasta, ingr=penne], NewMemory)
```

```
updateMemory(NewParams, OldMemory, NewMemory) :-
```

```
memory(OldMemory),
```

```
removeEmptyParams(NewParams, Params),
```

```
setof(Key, V^member(Key=V, Params), Keys),
```

```
remove(Keys, OldMemory, CleanedParams),
```

```
append(Params, CleanedParams, NewMemory).
```

[ingr=garlic]

[ingr]

[recipe=pasta]

[ingr=garlic, recipe=pasta]

NB: parameters on this slide are made up, just used here for the purpose of illustration!

Overview

- Dialogflow-MARBEL Connection
- **Patterns**
- Repair
- Responses
- Visuals
- Summary

Starting from human-human conversation

Should conversational agents be designed after human-human conversation?

- People are used to conversation
- People can optimize conversation in terms of efficiency and flexibility
- Natural conversation can lead to a good user experience

But:

- People know they are not talking to a human
- Conversational agents do not have the same capabilities as humans

Sequence organization (Schegloff, 2007)

Table 9.1 Anatomy of a conversation

Turns	Sequences	Activities
A: Hello	Greeting	Opening
B: Hi!	Greeting	
A: How's it going?	Welfare check	
B: Good	Welfare report	
A: Hey, are ya going tonight?	Pre-request	Requesting a ride
B: Yes	<i>Answer</i>	
A: Can I get a ride?	Request	
B: Sure!	Grant	
A: What time?	Inquiry	
B: Seven o'clock	Answer	
A: Okay.	<i>Acknowledgment</i>	
A: Thank you so much!	Pre-closing	Closing
B: No problem!	<i>Pre-closing</i>	
A: Bye	Farewell	
B: Buh bye	Farewell	

Conversation analysis

Study of how humans coordinate their conversations

- Turn-taking and turn conversational units (Sacks, Schegloff and Jefferson, 1974)
- Adjacency pairs (Schegloff and Sacks, 1973)
 - Greeting – greeting
 - Question – answer
 - Offer – accept/reject
 - Request – grant/deny

Sacks, H., Schegloff, E. A., & Jefferson, G. (1974). (1974). A simplest systematics for the organization of turn-taking in conversation. *Language*, 50, 696-735.

Schegloff, E. A., & Sacks, H. (1973). Opening up closings.

Conversational UX design (Moore & Arar, 2019)

- Grounded in conversation analysis
- Proposes pattern language to achieve **more natural** agent-user conversations, and **enhance re-usability**

Pattern A1.0	Inquiry (User)
	1 U: INQUIRY 2 A: ANSWER 3 U: SEQUENCE CLOSER
Example	1 U: What was the name of the first chatbot? 2 A: Her name was ELIZA 3 U: Ok

From conversation to patterns

Interaction for selecting a recipe

Agent: I can help you find a recipe. What kind of recipe are you looking for?

User: I would like to cook lemony spaghetti.

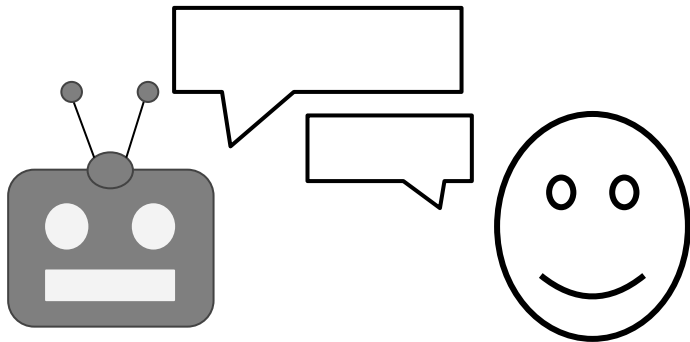
Agent: Lemony spaghetti is a great choice!

...



```
pattern([a50recipeSelect,  
  
        [agent, specifyGoal],  
  
        [user, recipeRequest],  
  
        [agent, recipeChoiceReceipt],  
  
        ...]).
```


Initiative



- Who determines the next conversation move?
- Control vs. flexibility
- Agent-initiated patterns have to be added to the **agenda** using the `PatternID`
- User-initiated patterns can be started by the user

Agenda

- High level structure of the interaction
- Includes agent-initiated patterns
- User and agent can add (sub)patterns to the interaction

Agenda for interaction



Greeting



Help find recipe



Confirm recipe



Farewell



(Sub)patterns

pattern([a50recipeSelect, [agent, specifyGoal], [user, recipeRequest], [agent, recipeChoiceReceipt],...]).

What if the user wants to deviate
from the conversation flow?



[user, addFilter]

Subpatterns

pattern([a50recipeSelect, [agent, specifyGoal], [user, recipeRequest], [agent, recipeChoiceReceipt],...]).

What if the user wants to deviate
from the conversation flow?

[user, addFilter]

We add a subpattern!

[a50recipeSelect, [agent, specifyGoal],

[a21featureRequest,

[user, addFilter],

[agent, removeConflicts(**Params**)],

[agent, ackFilter],...],

[user, recipeRequest], [agent, recipeChoiceReceipt],...]



**Agent keeps track of
the evolving
conversation using
the session/1
predicate**

Predicate: **session**([H | T])

H: where we are now in the
(*ongoing*) conversation

T: what was said and is part of
the *finished* conversation

Session updates (in linear order)

```
module updateSession(Actor, Intent, Params, Txt) {
```

```
  if session([ H | T ]) then {
```

```
    if H = [], matchingPattern(Actor, Intent, PatternId), log_info("session:", [ PatternId | T], Info)
```

```
      then delete(session([ [] | T])) + insert(session([ PatternId | T])) + updateSession(Actor, Intent, Params, Txt) + log(Info).
```

```
    if expectedIntent(Actor, Intent), session([H | T]), add(H, [Actor, Intent, Params], NewH), log_info("session: ", [ NewH | T], Info)
```

```
      then delete(session([ H | T])) + insert(session([ NewH | T])) + log(Info).
```

```
    if matchingPattern(Actor, Intent, PatternId), add(H, [PatternId], NewH), log_info("session: ", [NewH | T], Info)
```

```
      then delete(session([H | T])) + insert(session([NewH | T])) + updateSession(Actor, Intent, Params, Txt) + log(Info).
```

```
    if add(H, [b13], NewH), log_info("session: ", [ NewH | T], Info)
```

```
      then delete(session([H | T])) + insert(session([NewH | T])) + updateSession(Actor, Intent, Params, Txt) + log(Info).
```

```
  }}
```

Session updates - Case 1

```
module updateSession(Actor, Intent, Params, Txt) {  
  if session([ H | T ]) then {  
    if H = [], matchingPattern(Actor, Intent, PatternId), log_info("session:", [ [PatternId] | T], Info)  
    then delete(session([ [] | T])) + insert(session([ [PatternId] | T])) + updateSession(Actor, Intent, Params, Txt) + log(Info).  
  }  
}}
```

Intent initiates a new sequence

```
matchingPattern(Actor, Intent, PatternId) :-
```

```
  pattern([ PatternId, [Actor, Intent] | _ ], not(PatternId = b13), not(PatternId = b14).
```

Session updates - Case 2

```
module updateSession(Actor, Intent, Params, Txt) {  
  if session([ H | T ]) then {  
    if expectedIntent(Actor, Intent), session([H | T]), add(H, [Actor, Intent, Params], NewH), log_info("session: ", [ NewH | T], Info)  
    then delete(session([ H | T])) + insert(session([ NewH | T])) + log(Info).  
  }  
}
```

2

Intent contributes to current (ongoing) sequence

expectedIntent(Actor, Intent) :-

currentlyActiveSequence([PatternId | Sequence], getActorIntentPairs(Sequence, ActorIntentPairs),

pattern([PatternId | PatternSequence], **append**(ActorIntentPairs, [[Actor, Intent] | _], PatternSequence).

Session updates - Case 3

```
module updateSession(Actor, Intent, Params, Txt) {  
  if session([ H | T ]) then {  
    if matchingPattern(Actor, Intent, PatternId), add(H, [PatternId], NewH), log_info("session: ", [NewH | T], Info)  
    then delete(session([H | T])) + insert(session([NewH | T])) + updateSession(Actor, Intent, Params, Txt) + log(Info).  
  }  
}}
```

Starts a new subsequence

matchingPattern(Actor, Intent, PatternId) :-

```
pattern([ PatternId, [Actor, Intent] | _ ]), not(PatternId = b13), not(PatternId = b14).
```

Session updates - Case 4

```
module updateSession(Actor, Intent, Params, Txt) {  
  if session([ H | T ]) then {  
    if add(H, [b13], NewH), log_info("session: ", [ NewH | T], Info)  
      then delete(session([H | T])) + insert(session([NewH | T])) + updateSession(Actor, Intent, Params, Txt) + log(Info).  
  }  
}
```

Starts a repair (as a last resort, none of the other options worked)

Sequence completion

Check whether a **sequence** completes a **pattern**:

c10

[[c10, [agent,greeting,[],
[user,greeting,[],]]]

completedPattern([PatternId | Sequence]) :-

[[agent,greeting],
[user,greeting]]

getActorIntentPairs(Sequence, ActorIntentPairs),

[[agent,greeting],
[user,greeting]]

pattern([PatternId | PatternSequence]),

append(ActorIntentPairs, [], PatternSequence).

Current top level active pattern ID

session(
[

Current top level active (and incomplete) pattern

[**a50recipeName**, [agent,recipeInquiry,[],

[featureRequest,[user,addFilter,[ingredient=garlic]],

[agent,removeConflicts([ingredient=garlic]),[]],

[agent,ackFilter,[]]]

A pattern initiating a **subsequence**

],

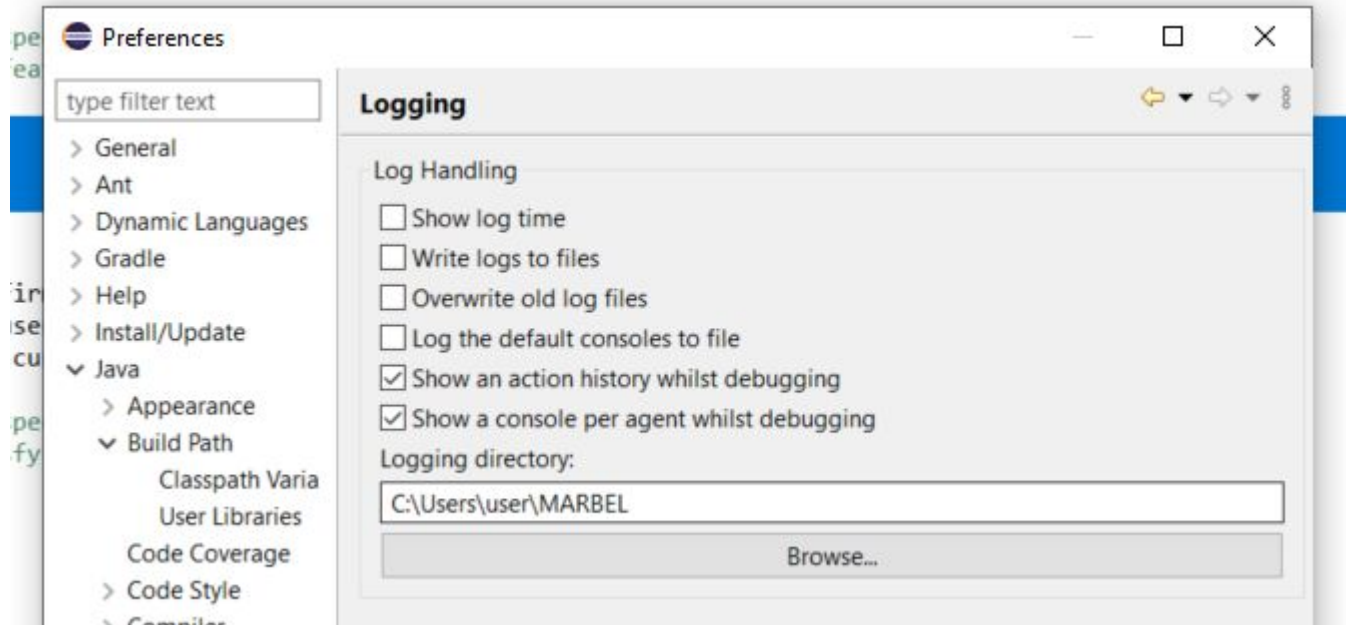
[c11,[agent,greeting,[]],[agent,selfIdentification,[]],[user,greeting,[]]]

]

)

Reading the session (use introspector or log files)

- MARBEL logs can be found in the folder specified under your Eclipse logging preferences



Reading the session

In the logs you should be able to find a session (a long string with nested patterns):

```
[[],[c40,[agent,sessionCloser,[],[agent,farewell,[]],[a50recipeConfirm,...],[a50recipeSelect,[agent,specifyGoal,[],[a21featureRequest,[user,addFilter,[ingredient=garlic],[agent,removeConflicts([ingredient=garlic]),[]],[agent,ackFilter,[]],[a21noMoreFilters,...],[user,recipeRequest],[agent,recipeChoiceReceipt]],c10,[agent,greeting,[]],[agent,selfIdentification,[],[user,greeting,[]],[start,[user,button,[button='Start']]]]]],
```

Let's take a closer look at the session!

Reading the session

1. Look for pattern IDs!

```
[[],  
[c40,[agent,sessionCloser,[],[agent,farewell,[]]],  
[a50recipeConfirm,...],  
[a50recipeSelect,[agent,specifyGoal,[],  
[a21featureRequest,[user,addFilter,[ingredient=garlic]],[agent,removeConflicts([ingredient=garlic]),[]],[agent,ackFilter,[]]],  
[a21noMoreFilters,...],  
[user,recipeRequest],[agent,recipeChoiceReceipt]],  
[c10,[agent,greeting,[],[agent,selfIdentification,[],[user,greeting,[]]],  
[start,[user,button,[button='Start']]]]]"
```

Reading the session

2. Identify subpatterns

```
[[],  
[c40,[agent,sessionCloser,[],[agent,farewell,[]]],  
[a50recipeConfirm,...],  
[a50recipeSelect,[agent,specifyGoal,[],  
  [a21featureRequest,[user,addFilter,[ingredient=garlic]],[agent,removeConflicts([ingredient=garlic]),[]],[agent,ackFilter,[]]],  
  [a21noMoreFilters,...],  
[user,recipeRequest],[agent,recipeChoiceReceipt]],  
[c10,[agent,greeting,[]],[agent,selfIdentification,[]],[user,greeting,[]]],  
[start,[user,button,[button='Start']]]]"
```


Reading the session

```
[[],  
[c40,[agent,sessionCloser,[],[agent,farewell,[]]], 7  
[a50recipeConfirm,...], 6  
[a50recipeSelect,[agent,specifyGoal,[], 3  
4 [a21featureRequest,[user,addFilter,[ingredient=garlic]],[agent,removeConflicts([ingredient=garlic]),[]],[agent,ackFilter,[]]],  
5 [a21noMoreFilters,...],  
[user,recipeRequest],[agent,recipeChoiceReceipt]],  
[c10,[agent,greeting,[]],[agent,selfIdentification,[]],[user,greeting,[]]], 2  
[start,[user,button,[button='Start']]]]" 1
```

3. Read patterns from end to beginning, and subpatterns from beginning to end.
Subpatterns are always initiated after their parent pattern has started.

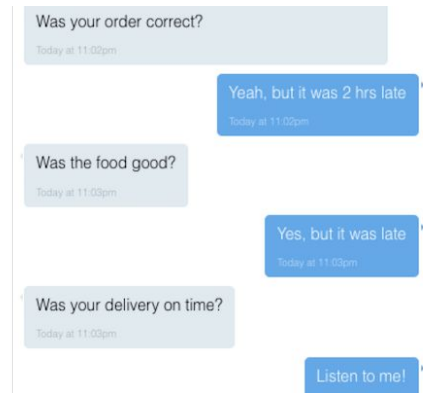
Overview

- Dialogflow-MARBEL connection
- Patterns
- **Repair**
- Responses
- Visuals
- Summary

What causes breakdown of human-chatbot conversations?

What causes breakdown of human-chatbot conversations?

- Misunderstanding
 - Human intent / entity not recognized
 - Agent utterance not understood
- Scripted agent flow
- External factors (noise, bad ASR)
- No repair facilitated
- ...



Repair strategies

Rapid reprompt: “Could you say that again, please?”

Particular repair prompt: “How many people did you say?”

Escalating detail: “So you want to book a table for 3 persons?”

Repair pattern, agent-initiated (Moore & Arar, 2019)

Pattern B1.2.0 Paraphrase Request (Agent).

1 A: <ANY UTTERANCE>
2 U: <UNKNOWN UTTERANCE>
3 A: INITIAL PARAPHRASE REQUEST
4 U: <KNOWN PARAPHRASE>
5 A: <APPROPRIATE RESPONSE>

Example 6.2 Paraphrase Request (Agent).

1 A: Hello. How are you?
2 U: **bitchin'!**
3 A: What do you mean?
4 U: **I'm great!**
5 A: That's great!

Pattern B1.2.1 Subsequent Paraphrase Requests.

1 A: <ANY UTTERANCE>
2 U: <UNKNOWN UTTERANCE>
3 A: INITIAL PARAPHRASE REQUEST
4 U: <UNKNOWN UTTERANCE>
5 A: SUBSEQUENT PARAPHRASE REQUEST

Example 6.3 Subsequent Paraphrase Requests.

1 A: How can I help you?
2 U: **what's in theaters?**
3 A: What do you mean?
4 U: **what movies are currently playing?**
5 A: I'm sorry, I still don't understand.

Repair pattern, user-initiated (Moore & Arar, 2019)

Pattern B2.4.0 Paraphrase Request.

- 1 A: <ANY UTTERANCE>
- 2 U: PARAPHRASE REQUEST
- 3 A: PARAPHRASE

Example 6.13 Paraphrase Request (elaboration).

- 1 A: I guess I like movies with a strong AI lead.
- 2 U: **what do you mean?**
- 3 A: I enjoy movies in which the main character is an Artificial Intelligence.

Considerations for repair patterns

- When should repair be initiated?
 - Non-understanding
 - Pattern mismatch
 - ...
- How should the repair be phrased?
- Can the visuals support the repair?



Overview

- Dialogflow-MARBEL connection
- Patterns
- Repair
- **Responses**
- Visuals
- Summary

Which **response** of Weatherbot would you prefer?

A) *U: Hey WeatherBot! Could you tell me the weather?*

WB: The weather in Amsterdam is chilly at 4 degrees Celsius, with showers expected around noon. Would you like to hear some more?

B) *U: Hey WeatherBot! Could you tell me the weather?*

WB: Yes.

C) *U: Hey WeatherBot! Could you tell me the weather?*

WB: The weather in Amsterdam is chilly at 4 degrees Celsius, with showers expected around noon. The humidity is 80 percent and wind speed is 9 km/h. In the nighttime, temperatures will drop to 0 degrees. Tomorrow the weather in Amsterdam is chilly at 3 degrees Celsius. The humidity is 75 percent and wind speed is 7 km/h. In the nighttime, temperatures will drop to 1 degree.

Maxime of quantity (Grice, 1975)

1. Make your contribution **as informative as is required**
2. Do **not make your contribution more informative** than is required.

Patterns, intents, and responses

```
[c10,[agent,greeting,[]],[agent,selfIdentification,[]],[user,greeting,[]]]
```

A pattern specifies the dialog moves (intents) of agents and users,
but NOT the natural language responses.

Specify **agent** (text) responses in the **responses.pl** file for each intent using **text/2**.

Greetings

Provide multiple examples, so the agent can **VARY** the NLG for its **greeting** intent:

```
text(greeting, "Hey there!").
```

```
text(greeting, "How are you doing?").
```

```
text(greeting, "Nice to see you!").
```



You can be **creative** and go beyond the example: Use ChatGPT or any other tool.

Varying your agent's responses

You only need to specify a variety of responses; random selection has already been defined in the **responses.pl** file:

```
text_generator(Intent, SelectedText) :-  
    currentTopLevel(PatternId),  
    findall(Text, text(PatternId, Intent, Text), Texts),  
    random_select(SelectedText, Texts, _).
```

Using knowledge the agent has

Example: **selfIdentification** intent

The agent in its database has an **agentName(Cookpanion)** fact (if you define it!!). This is knowledge the agent can use to generate its NLG response:

```
text(selfIdentification, Txt) :-
```

```
    agentName(Name), string_concat("I am ", Name, Txt).
```

Use
string_concat
for combining
strings for responses

Knowledge sources your agent has

A few sources that you could use:

- **Domain knowledge**, e.g. the `recipe_database.pl`, `dialogflow.pl`, `recipe_selection.pl` and `ingredient_hierarchies.pl` files.
- The dialog manager's **session history** and **memory**, constructed while performing dialog moves using the received intents and entity values.

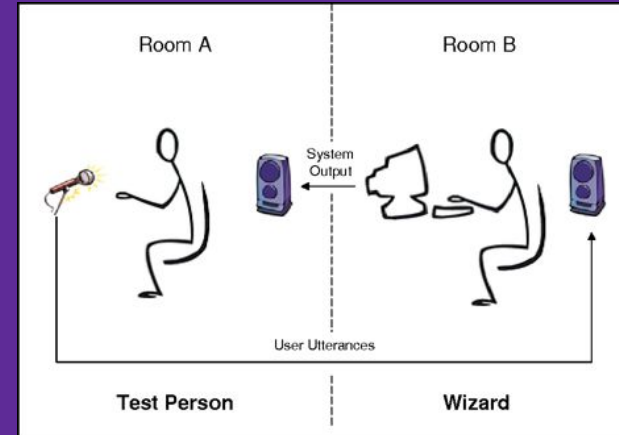
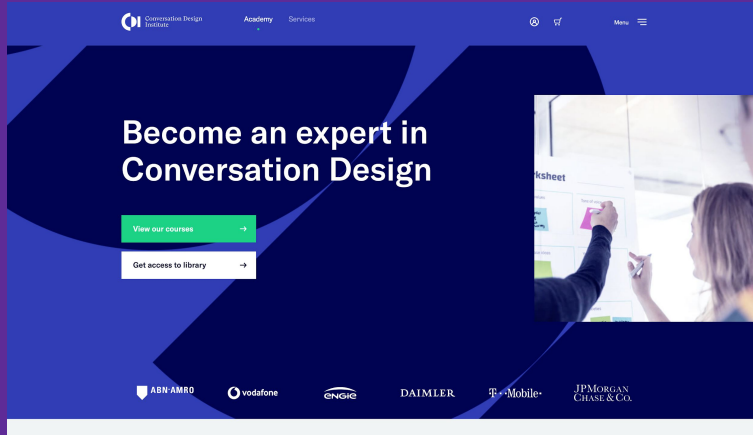
Domain knowledge

This knowledge could, e.g., be used in `responses.pl` to generate an ingredients availability check:

```
text(ingredientsCheck, Txt) :-  
    currentRecipe(Recipe), ingredientsList(Recipe, IngredientList),  
    string_concat("Do you have all the ingredients needed?",  
        IngredientList, Txt).
```

NB: not a very good example. Why not?

Response / conversation design



Some other design concepts

- Communicability
 - How does the user know what it can say / ask?
- User persona
 - Who will be making use of the agent (intended audience)? How does this affect the design?
- Agent persona
 - Making sure the agent has a consistent personality → language and communication style
- The “Happy Flow” (conversation design)
 - Designing for the conversations as you would expect the agent and user to have
- Fine-tuning conversation design
 - Long tail of directions that a conversation can take; what to address?

Styling your agent's responses: persona example

Design choice: recipe recommendation for the **elderly**.

Based on literature (*motivate* your choices in report!!):

- Register (choice of words): polite, no joking
- Conversation:
 - Offering assistance
 - Responding properly to stories
- Functionality:
 - Filtering for traditional recipes
 - Offering alternatives that cost less effort

Check your responses for being...

- **inclusive** (no “bro” please),
- **exhaustive** (different conversation paths, cases, situations),
- **consistent** (style, etc.)

These aspects are related to your design choices too, so **motivate in your report!!**

Overview

- Dialogflow-MARBEL Connection
- Patterns
- Repair
- Responses
- **Visuals**
- Summary

Visuals for supporting conversation

- Conversation structure
- Cognitive load
- Agent persona


```
<!DOCTYPE html>
<html>
<head>
  ... bootstrap.min.css ... <!-- WE ENABLE BOOTSTRAP 4 -->
  <title>Project Conversational Agents</title>
</head>
<body>

<YOUR HTML CODE GOES HERE>

</body>
</html>
```

Example HTML code generation

Some useful links:

- <https://www.w3schools.com/html/default.asp>
- <https://www.w3schools.com/bootstrap4/default.asp>

Find an HTML element you want to use

Check out e.g.: https://www.w3schools.com/bootstrap4/bootstrap_cards.asp

Example: (Simple) Bootstrap card:

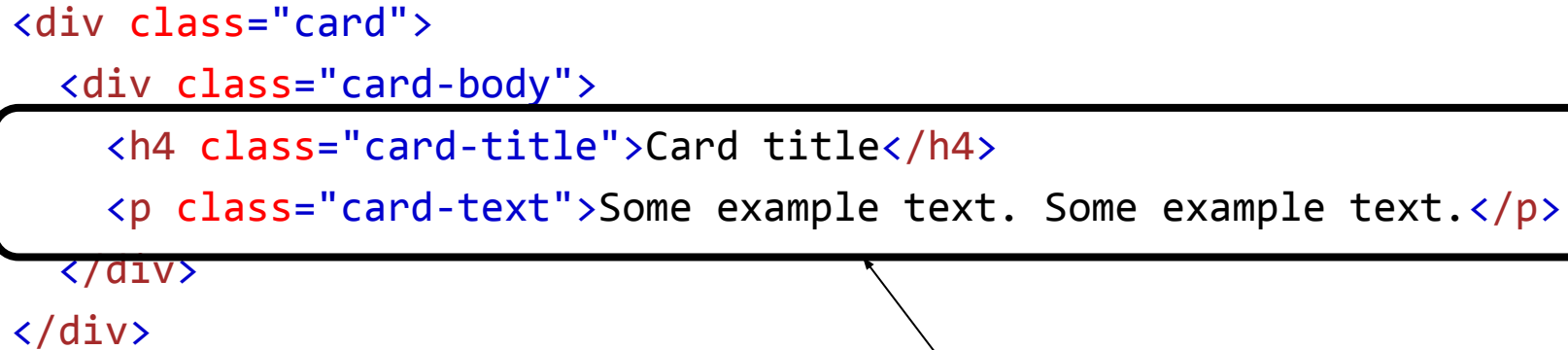
```
<div class="card">
  <div class="card-body">
    <h4 class="card-title">Card title</h4>
    <p class="card-text">Some example text. Some example text.</p>
  </div>
</div>
```

Work from the “inside out”

Check out e.g.: https://www.w3schools.com/bootstrap4/bootstrap_cards.asp

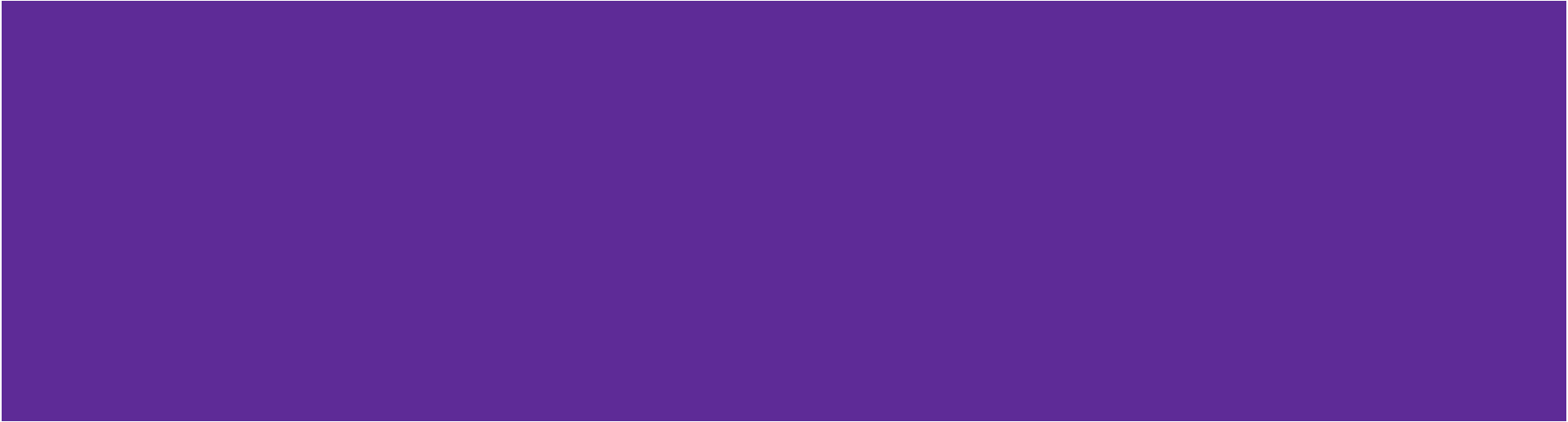
Example: (Simple) Bootstrap card:

```
<div class="card">  
  <div class="card-body">  
    <h4 class="card-title">Card title</h4>  
    <p class="card-text">Some example text. Some example text.</p>  
  </div>  
</div>
```



Start with creating the **h4** and **p** elements

Styling your pages



Styling your pages: Persona example

Design choice:

Styling for intended audience: recipe recommendation for the **elderly**

Based on literature (*motivate* your choices in report!!): “**Minimal styling**”

- Simple colors
- Only relevant images, and images would work in black and white as well as in color
- Smooth edges and line
- Only necessary icons/text/images on screen
- Sufficient contrast for legibility
- Making the application understandable, unambiguous and informative.

Overview

- Architecture
 - Dialogflow-MARBEL Connection
 - Patterns
 - Repair
 - Responses
 - Visuals
 - **Summary**
-

Summary

This week: *Check out the [course schedule \(LINK\)](#)!*

- Prepare Check-In Presentation for Friday session (see Course schedule)
- End of this week: aim for finishing implementation of all capabilities (patterns)!